

Payment System DB Design

Normalization Analysis — 1NF through BCNF

1. First Normal Form (1NF)

Definition: A relation is in First Normal Form (1NF) if all attributes are atomic (indivisible) and there are no repeating groups or multi-valued attributes.

Example 1: Group_Member — Multi-valued user/customer references

Before 1NF, Group_Member stored a set of user_id and customer_id values together in a single row, making the data non-atomic.

Fix: Separated each member relationship into individual rows with a composite key (group_id, user_id), ensuring each row holds exactly one member entry.

Example 2: Split_Expense — Multi-valued beneficiary references

Before 1NF, Split_Expense stored multiple beneficiary_id values within a single record, representing several payees for one expense.

Fix: Each beneficiary share is now a separate row in Split_Expense with its own split_id, beneficiary_id, and share_amount.

Example 3: Transaction — Category stored as repeating codes

Before 1NF, Transaction embedded multiple category_name values as a comma-separated string in one column.

Fix: Extracted categories into a dedicated Category table with category_class as the primary key. Transaction.category_name now holds a single atomic foreign key reference.

Result: All attributes are atomic; multi-valued data moved to separate tables. Schema satisfies 1NF.

2. Second Normal Form (2NF)

Definition: A relation is in Second Normal Form (2NF) if it is already in 1NF and every non-key attribute is fully dependent on the entire primary key, not just part of it.

Example 1: Group_Member table

Before 2NF, Group_Member had a composite primary key (group_id, user_id) but stored joined_at and customer_id — attributes that depended only on user_id, not the full composite key.

Functional Dependencies:

user_id	customer_id (partial dependency — violates 2NF)
(group_id, user_id)	joined_at (full dependency — correct)

Fix: Ensured customer_id is managed in the Customer table and that Group_Member only stores attributes fully dependent on (group_id, user_id)

Example 2: Split_Expense table

Before 2NF, Split_Expense included share_amount and status — some attributes that were only partially dependent on group_id rather than the full key.

Functional Dependencies:

group_id	group-level info (partial) — moved to Split_Group
transaction_id	transaction details (partial) — moved to Transaction
(split_id)	share_amount, status, paid_by, beneficiary_id (full)

Fix: Promoted split_id as sole primary key of Split_Expense. Group-level and transaction-level data is accessed via foreign keys to Split_Group and Transaction.

Result: We split tables so that attributes depend on the full key. Schema satisfies 2NF.

3. Third Normal Form (3NF)

Definition: A relation is in Third Normal Form (3NF) if it is already in 2NF and has no transitive dependencies (non-key attributes depending on other non-key attributes).

Example 1: Merchant table — transitive dependency through user_id

Before 3NF, Merchant contained user_id, name, contact_number, and business_domain. User-level profile data was transitively dependent through user_id.

Functional Dependencies:

merchant_id	user_id (direct)
user_id	email, password, role (transitive — stored in User)
merchant_id	name, contact_number, business_domain (direct)

Fix: Separated user credentials and role into the User table. Merchant retains only merchant-specific attributes and references User via user_id foreign key.

Example 2: Transaction table — category details

Before 3NF, Transaction stored category_name and an inline description of that category. The description was transitively dependent: transaction_id → category_name → description.

Functional Dependencies:

transaction_id	category_name (direct)
category_name	description (transitive — violates 3NF)

Fix: Extracted category data into the Category table with category_class and description. Transaction references Category via foreign key.

Result: No transitive dependencies remain. Schema satisfies 3NF.

4. Boyce–Codd Normal Form (BCNF)

Definition: A relation is in BCNF if it is already in 3NF and for every functional dependency $X \rightarrow Y$, X is a superkey.

Example 1: Scheduled_Payment — non-superkey determinant

Before BCNF, Scheduled_Payment included both customer_id and beneficiary_id. Since beneficiary_id determines bank_customer_id (a banking reference), and beneficiary_id is not a superkey of Scheduled_Payment, this violated BCNF.

Functional Dependencies:

schedule_id	customer_id, beneficiary_id, amount, frequency, ... (superkey — OK)
beneficiary_id	bank_customer_id, beneficiary_name (not a superkey — violates BCNF)

Fix: Decomposed into Scheduled_Payment and a separate Beneficiary table. Beneficiary owns beneficiary_id as its primary key and holds bank_customer_id and beneficiary_name. Scheduled_Payment references Beneficiary via foreign key.

Example 2: Bank_Account — user dependency

Before BCNF, Bank_Account held user_id and account details. Since user_id determines user profile fields that were also stored in User, user_id acted as a non-superkey determinant within the bank account context.

Functional Dependencies:

account_number	user_id, bank_name, balance, is_primary (superkey — OK)
user_id	email, role (non-superkey in Bank_Account — violates BCNF)

Fix: All user profile attributes are exclusively in the User table. Bank_Account only stores banking-specific attributes and references User via user_id foreign key with no duplicated user data.

Result: After decomposition, every determinant is a superkey. Candidate key overlaps removed. Schema satisfies BCNF.

Normalization Summary

Normal Form	Condition	Issue Found	Resolution
1NF	Atomic attributes	Multi-valued: Group_Member, Split_Expense, Category	Separate rows / dedicated tables
2NF	Full key dependency	Partial deps in Group_Member, Split_Expense	Promoted single PKs; moved partial data to parent tables
3NF	No transitive deps	Merchant user data, Transaction category description	Extracted User, Category into own tables
BCNF	Every determinant is a superkey	Beneficiary in Scheduled_Payment, user fields in Bank_Account	Decomposed into Beneficiary, kept User separate